

MAZE SOLVER USING A*

Informed Search – Individual Technical Record

Project Objective

Build a grid maze solver using the A* informed-search algorithm with the Manhattan-distance heuristic. The system should find a path from the start cell to the goal cell, count the nodes expanded by A*, and compare its performance with Breadth-First Search (BFS) on the same maze.

1. ABSTRACT

Maze solving is a common application of Artificial Intelligence search algorithms. This project implements A* search on a grid maze where 0 represents an open path and 1 represents a wall. A* combines the cost already travelled with an estimated remaining cost using the Manhattan-distance heuristic. BFS is also implemented on the same maze so that the number of expanded nodes can be compared. For the given maze, both algorithms produce the same shortest path, while A* expands fewer nodes.

2. INTRODUCTION

A* is an informed search algorithm that selects the next node using an evaluation value $f(n) = g(n) + h(n)$. Here, $g(n)$ is the actual path cost from the start node to n , while $h(n)$ is the estimated cost from n to the goal. For a four-directional grid with unit movement cost, Manhattan distance is an appropriate heuristic because it estimates the minimum number of horizontal and vertical moves required to reach the goal.

3. PROBLEM STATEMENT

Develop a Python program that solves a fixed grid maze from a specified start position to a specified goal position using A*. The program must implement the Manhattan-distance heuristic, count nodes expanded by A*, and compare the result with BFS on the same maze.

4. OBJECTIVES

- Represent a maze using a two-dimensional grid.
- Identify valid neighbouring cells while avoiding walls and boundaries.
- Implement Manhattan-distance heuristic for A*.
- Find the path from start to goal using A*.
- Count the nodes expanded during A* search.
- Implement BFS and solve the same maze.
- Compare the path cost and nodes expanded by A* and BFS.

5. MAZE REPRESENTATION

The maze uses 0 for an open cell and 1 for a wall. The start is (0, 0) and the goal is (5, 5).

S(0,0)	0	0	0	1	0
1	1	0	0	1	0
0	0	0	1	0	0
0	1	1	1	0	1
0	0	0	0	0	0
1	1	1	1	1	G(5,5)

6. A* EVALUATION FUNCTION

$$f(n) = g(n) + h(n)$$

$g(n)$ = cost from the start node to the current node.

$h(n)$ = Manhattan-distance estimate to the goal.

For a node (r,c) and goal (rg,cg) : $h(n) = |r - rg| + |c - cg|$.

7. WORKED HEURISTIC EXAMPLE

For the start node $(0,0)$ and goal $(5,5)$:

$$h(0,0) = |0-5| + |0-5| = 10.$$

$$\text{For node } (0,1): h(0,1) = |0-5| + |1-5| = 9.$$

Thus, the heuristic estimates how many grid moves remain when only four-directional movement is allowed.

8. A* ALGORITHM / PROCEDURE

1. Initialize a priority queue with the start node.
2. Set the start cost to 0 and its parent to None.
3. Remove the node having the smallest f-cost from the priority queue.
4. Increase the expanded-node counter.
5. If the current node is the goal, stop the search.
6. Generate valid neighbouring cells.
7. Calculate the new path cost for every neighbour.
8. If the new cost is better, update its cost, parent and f-cost, then add it to the priority queue.
9. Reconstruct the path by following parent links from goal to start.
10. Return the path and number of expanded nodes.

9. BFS ALGORITHM / PROCEDURE

BFS starts from the same start node and explores cells level by level using a queue. Each unvisited valid neighbour is added to the queue and its parent is stored. When the goal is reached, the path is reconstructed through the parent dictionary. The expanded-node counter is used for a direct comparison with A*.

10. IMPLEMENTATION

The implementation is written in Python. `heapq` is used as the priority queue for A*, while `deque` is used as the queue for BFS. Dictionaries store path costs and parent relationships. Every legal movement has a cost of 1.

11. SAMPLE OUTPUT

Running the supplied program on the given 6×6 maze produces the following results:

Algorithm	Path Cost	Nodes Expanded
A*	14	17
BFS	14	19

A* Path: $(0,0) \rightarrow (0,1) \rightarrow (0,2) \rightarrow (1,2) \rightarrow (2,2) \rightarrow (2,1) \rightarrow (2,0) \rightarrow (3,0) \rightarrow (4,0) \rightarrow (4,1) \rightarrow (4,2) \rightarrow (4,3) \rightarrow (4,4) \rightarrow (4,5) \rightarrow (5,5)$

The path contains 15 cells, so the path cost is 14 moves. BFS finds the same path cost but expands 19 nodes, compared with 17 nodes expanded by A*.

12. COMPARISON OF A* AND BFS

Feature	A*	BFS
---------	----	-----

Search type	Informed search	Uninformed search
Main data structure	Priority queue	Queue
Heuristic	Manhattan distance	None
Path cost	14	14
Nodes expanded	17	19

For this particular maze, A* expands 2 fewer nodes than BFS while producing the same path cost. This demonstrates how heuristic information can guide search toward the goal.

13. RESULTS

The A* implementation successfully finds a valid path from the start to the goal. The Manhattan heuristic guides the priority-queue search using both the distance already travelled and the estimated remaining distance. On the supplied maze, A* expands 17 nodes and BFS expands 19 nodes. Both return a path with cost 14.

14. APPLICATIONS

- Robot path planning and navigation.
- Game character movement and map navigation.
- Route planning in grid-based environments.
- Puzzle and maze solving.
- Autonomous navigation systems.

15. LIMITATIONS

The current program uses a fixed maze and unit movement cost. It allows movement only in four directions and does not include diagonal movement. The comparison is based on this single maze, so the node-expansion difference should not be treated as a general performance result for every maze.

16. CONCLUSION

The project demonstrates an informed-search approach to maze solving using A*. By combining actual path cost with the Manhattan-distance heuristic, A* finds the shortest path in the given four-directional unit-cost maze. The BFS comparison shows that both algorithms obtain a path cost of 14, while A* expands 17 nodes compared with 19 for BFS on the supplied maze. The nodes-expanded counter provides a simple way to observe the effect of informed search.

17. REFERENCES

1. Stuart Russell and Peter Norvig, Artificial Intelligence: A Modern Approach.
2. Python Documentation – heapq module.
3. Python Documentation – collections.deque.
4. Project specification: Maze Solver using A* – Informed Search.

APPENDIX - PYTHON IMPLEMENTATION

The following is the Python implementation supplied for the project.

```
from collections import deque
import heapq

# 0 = open path, 1 = wall
maze = [
    [0, 0, 0, 0, 1, 0],
    [1, 1, 0, 0, 1, 0],
    [0, 0, 0, 1, 0, 0],
    [0, 1, 1, 1, 0, 1],
    [0, 0, 0, 0, 0, 0],
    [1, 1, 1, 1, 1, 0]
]

start = (0, 0)
goal = (5, 5)

# Manhattan Distance Heuristic
def heuristic(node, goal):
    return abs(node[0] - goal[0]) + abs(node[1] - goal[1])

# Get valid neighbouring cells
def get_neighbors(node):
    r, c = node
    directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]

    for dr, dc in directions:
        nr, nc = r + dr, c + dc

        if (0 <= nr < len(maze) and
            0 <= nc < len(maze[0]) and
            maze[nr][nc] == 0):
            yield (nr, nc)

# A* Algorithm
def astar():
    priority_queue = []
    heapq.heappush(priority_queue, (0, start))

    cost = {start: 0}
    parent = {start: None}
    expanded = 0

    while priority_queue:
        f, current = heapq.heappop(priority_queue)

        expanded += 1

        if current == goal:
            break

        for neighbor in get_neighbors(current):
            new_cost = cost[current] + 1

            if neighbor not in cost or new_cost < cost[neighbor]:
                cost[neighbor] = new_cost
                f_cost = new_cost + heuristic(neighbor, goal)

                heapq.heappush(
                    priority_queue,
                    (f_cost, neighbor)
                )

                parent[neighbor] = current

    path = []

    if goal in parent:
        current = goal

        while current is not None:
            path.append(current)
            current = parent[current]

        path.reverse()

    return path, expanded

# BFS Algorithm
def bfs():
    queue = deque([start])
    parent = {start: None}
    expanded = 0
    while queue:
```

```

    current = queue.popleft()
    expanded += 1

    if current == goal:
        break

    for neighbor in get_neighbors(current):
        if neighbor not in parent:
            parent[neighbor] = current
            queue.append(neighbor)

    path = []

    if goal in parent:
        current = goal

        while current is not None:
            path.append(current)
            current = parent[current]

        path.reverse()

    return path, expanded

# Run A*
astar_path, astar_nodes = astar()

# Run BFS
bfs_path, bfs_nodes = bfs()

# Display Results
print("----- A* SEARCH -----")
print("Path:", astar_path)
print("Path Cost:", len(astar_path) - 1)
print("Nodes Expanded:", astar_nodes)

print("\n----- BFS SEARCH -----")
print("Path:", bfs_path)
print("Path Cost:", len(bfs_path) - 1)
print("Nodes Expanded:", bfs_nodes)

print("\n----- COMPARISON -----")
print("A* Nodes Expanded :", astar_nodes)
print("BFS Nodes Expanded:", bfs_nodes)

```